Why the Path Matters

A visual account of the tombstone-free, path-carrying RGA

Sal / FP Launchpad

Abstract

The replicated growable array (RGA) in `RGA_Tombstone_Free` stores only *live* elements: a delete physically removes its target, leaving no tombstone. To stay convergent without tombstones, each operation carries the *ancestor path* of the node it references. This note explains, with pictures, two things: **(i) why the implementation converges** — the three-way merge recovers a concurrently-deleted node's parent from the lowest common ancestor (LCA); and **(ii) why the path is necessary *for the single-replica commutation proof*** — to discharge the framework's do-commutation VC, where there is no LCA, an insert whose anchor was deleted has nowhere to recover its position *unless it carries that position itself*. The qualification matters: the merge alone already converges the concurrent case *without* any path (`merge_converges_concurrent`); it is this commutation *obligation*, not operational convergence, that the path serves. We give the counterexample that diverges without the path, the fix that the path provides, and the asymmetry that makes the path mandatory for Ins but optional for Del. Every claim here is mechanised in Lean (§7).

1 The data type: a forest of live records

The state is a finite map $\sigma : \mathrm{id} \mapsto (\mathsf{el}, \mathsf{anc})$ from an identity to its element and its *immediate anchor* — the identity it was inserted after. Identity 0 is the root sentinel and is never stored. So a state *is* a forest rooted at 0: an edge from a node to its anchor (Figure 1). We write $\mathsf{ids}(\sigma)$ for the stored identities, $\mathsf{el}(\sigma, t)$ and $\mathsf{anc}(\sigma, t)$ for the element and anchor of t, and $\mathsf{contains}(\sigma, t)$ for "t is live". Records use the reference notation (id, element, anchor).

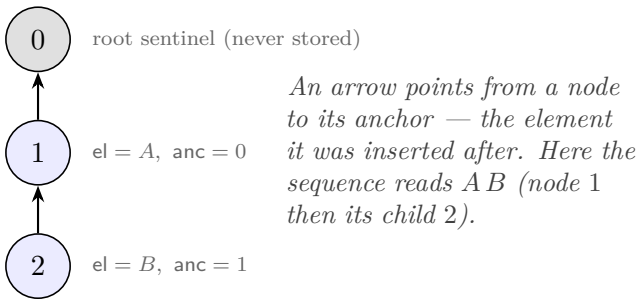


Figure 1: The state as a forest. $\sigma = \{(1, A, 0), (2, B, 1)\}$.

Operations. There are two, and (crucially) each carries a *path* p — the ancestor chain of its referenced node, nearest first:

- $\mathsf{Ins}(e, p, a)$ at a fresh id t: insert e anchored at a. It stores $(t, e, \mathsf{resolve}(\sigma, a :: p))$, where resolve returns the first *live* candidate in its list, else 0. When a is live this is just a; when a is dead it climbs p to the nearest live ancestor.

- $\mathsf{Del}(p, x)$: *splice* out x — physically remove it and re-parent its children to $\mathsf{resolve}(\sigma, p)$, the nearest live ancestor of x. No tombstone is kept.

Figure 8 shows the defining move of a tombstone-free delete: the splice. Deleting 2 removes it and lifts its child 3 one level, onto 2's parent 1.

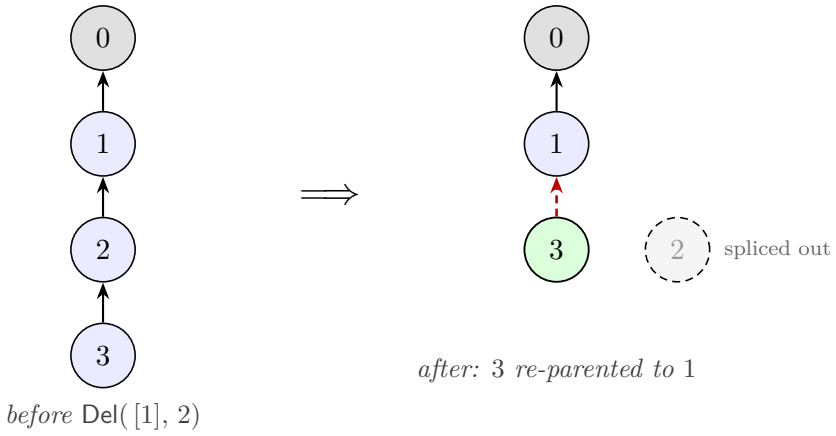


Figure 2: The splice. A tombstone-free $\mathsf{Del}(2)$ physically removes 2 and rehomes its child 3 to 2's live parent 1 (red dashed = the new anchor). Compare a tombstone design, which would instead keep 2 as a dead marker.

2 Why the implementation converges: the LCA does the remembering

Concurrency is reconciled by a three-way merge $\mathsf{merge}(L, A, B)$ where L is the least common ancestor of branches A and B. It has two steps: *survival* (an OR-set merge on the three id sets) and *re-parenting* (climb each survivor's birth-anchor up L's chain). The survivor set is

$$I \;=\; \big(\mathsf{ids}(L) \cap \mathsf{ids}(A) \cap \mathsf{ids}(B)\big) \;\cup\; \big(\mathsf{ids}(A) \setminus \mathsf{ids}(L)\big) \;\cup\; \big(\mathsf{ids}(B) \setminus \mathsf{ids}(L)\big)$$

— an id lives iff it is kept on both sides, or freshly added on either; equivalently, a node lives unless it was in L and deleted on a side. Each survivor t takes its element $\mathsf{el}(t)$ and *birth-anchor* $\beta(t)$ from L if present, else A, else B; the birth-anchor is then climbed up L's chain — via L's anchor map anc_L — to the nearest live survivor:

$$\mathsf{climb}_I(a) \;=\; \begin{cases} a, & a = 0 \text{ or } a \in I, \\ \mathsf{climb}_I(\mathsf{anc}_L(a)), & a \in \mathsf{ids}(L) \setminus I. \end{cases}$$

$$\mathsf{merge}(L, A, B) \;=\; \big\{\, (t,\ \mathsf{el}(t),\ \mathsf{climb}_I(\beta(t))) \;\big|\; t \in I \,\big\}.$$

The point for convergence is the phrase **"up L's chain"**: even when a node was deleted on one branch, *the LCA still holds it live, with its parent.* So the merge can always recover where a survivor should attach — it reads parents from L, and never needs the operations' paths. Figure 3 is the canonical case: A inserts 3 after 2 while B concurrently deletes 2; the merge climbs 3's dead birth-anchor 2 up the LCA chain to the live 1.

This is the whole reason a tombstone-free RGA can converge at all: the merge has a third input, L, that retains the deleted node's ancestry. Keep that fact in mind — the next section is about the one place where there is *no* such third input.

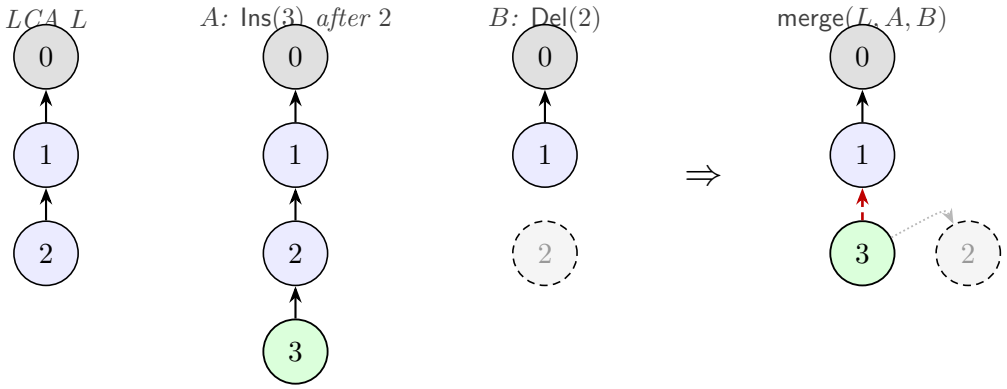
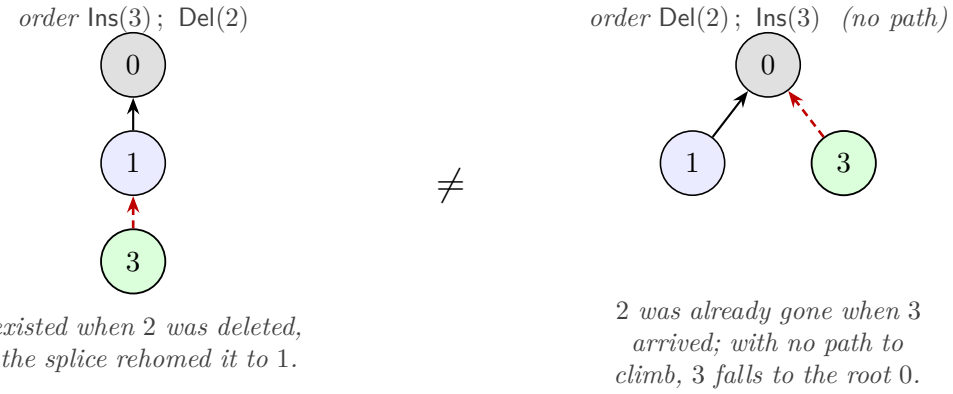


Figure 3: Why merge converges. 3 was born under 2 (dotted, 2's ghost), but 2 is deleted on B. The merge climbs 3's birth-anchor up the LCA chain — $2 \in \mathsf{ids}(L) \setminus I$, so $\mathsf{climb}_I(2) = \mathsf{climb}_I(\mathsf{anc}_L(2)) = \mathsf{climb}_I(1) = 1$ — and rehomes 3 to 1 (red dashed). The LCA is where "2's parent is 1" is read. **No path on the operations is used.**

3 Why the path is necessary: a single replica has no LCA

Convergence of a replicated data type also requires that operations applied in different orders on *one* replica agree — the framework discharges this as a *commutation* condition on do (single-replica application). And here is the catch: do **has no LCA**. There is only the current state. So when an insert's anchor has already been deleted, the parent the merge would have read from L is simply *gone* — unless the insert brought it along.

The counterexample (no path). Take $\sigma = \{(1, A, 0), (2, B, 1)\}$ and the two operations $\mathsf{Ins}(C, 2)$ (insert C as id 3 after 2) and $\mathsf{Del}(2)$. Apply them in both orders, with a *prefix-free* insert (anchor only, no path):



3 *existed when* 2 *was deleted, so the splice rehomed it to* 1.

2 *was already gone when* 3 *arrived; with no path to climb,* 3 *falls to the root* 0.

Figure 4: **Divergence without the path.** The same insert/delete pair, two orders, two different trees: 3 is anchored at 1 on the left but at 0 on the right. The information "2's parent is 1" was destroyed by the tombstone-free delete, and nothing on the right carries it. Mechanised as `prefixfree_diverges`.

The two orders disagree on 3's anchor (1 versus 0): the design does *not* commute. Because the delete physically removed 2, the fact "$\mathsf{anc}(2) = 1$" survives nowhere on the right — and a prefix-free

insert has no way to reconstruct it.

The fix (carry the path). Give the insert the ancestor path of its anchor, $\mathsf{Ins}(C, [1], 2)$. Now when 2 is dead, resolve climbs the carried path [1] to the live 1, and both orders land 3 under 1 (Figure 5). The path is exactly the ancestry that the tombstone-free state threw away; carrying it on the operation restores commutation.

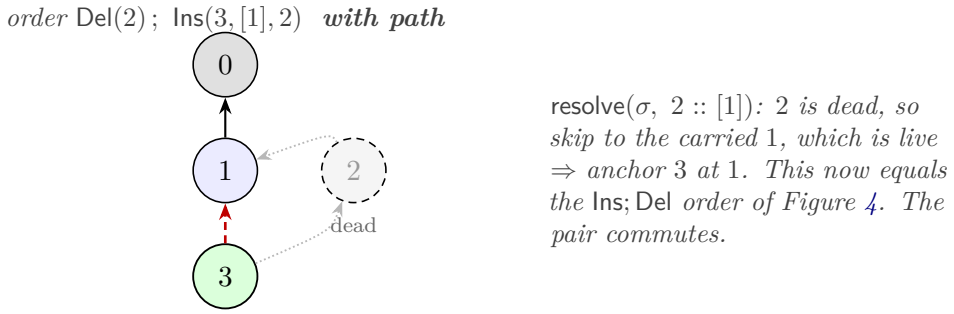


Figure 5: **The path restores convergence.** The carried path [1] lets the late insert climb past the deleted 2 to the live 1, matching the other order. Mechanised as `prefix_converges`.

The slogan. *A tombstone keeps the deleted node's parent in the **state**; the path keeps it on the **operation**.* Remove the tombstone and that parent has to ride along on any operation that might still need it — and on a single replica (no LCA) the insert is exactly such an operation. This is also why the two are *mutually exclusive but jointly unavoidable for the proof*: to satisfy the do-commutation VC you may drop the tombstone or drop the path, but not both. Dropping both defeats that route — provably (`RGA_PrefixFree_Impossible.lean`); the runnable witness that drops both yet still converges *via merge* is `RGA_PrefixFree_Impl.lean`.

4 The path is ghost state: needed for the proof, not for execution

The previous section's "necessary" is a statement about the *proof obligation*, and its scope is worth being exact about: in a real execution the path is **never read**. It is *ghost state* — pure proof scaffolding that a runtime implementation can drop entirely.

Why it is never read comes down to how operations are generated. A client issues an operation against **its own local replica**, on the state it currently sees; you insert after, or delete, a node that is *live in front of you*. On any state where the anchor is live, `resolve` short-circuits on that anchor and never inspects the prefix — exactly what `ins_path_free` and `del_path_free` prove. So for every operation a client can actually *issue*, the path is dead code. The one state that reads it — an insert whose anchor was *already deleted on the same replica* — is manufactured only by the do-commutation VC, which quantifies over *all* states and, in forcing the two orders to agree, constructs exactly that intermediate. It sits **outside the reachable set** (Figure 6).

Why the system model rules it out. Merge here is **voluntary**, pull-based — like `git pull`. A replica's state does not change under its feet: remote updates are incorporated only when the replica *chooses* to merge, at well-defined points, never in the middle of forming an operation. So the anchor a client references is live when the operation is issued and stays live until it is applied locally; the dead-anchor state never arises on the replica that generated the op. And even in a

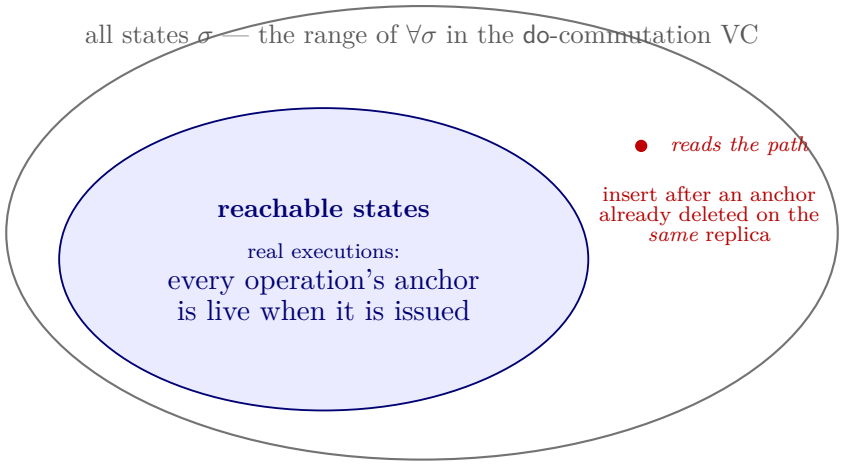


Figure 6: The path is ghost state. The do-commutation VC ranges over *all* states; real executions live in the shaded *reachable* region, where every operation's anchor is live and `resolve` short-circuits before the prefix (`ins_path_free` / `del_path_free`). The path is consulted only at the red state — an insert whose anchor was already deleted on its own replica — which a voluntary, pull-based merge never produces.

model where remote state *could* interleave, the client can **reject an operation whose anchor is no longer present** — a cheap local check — so the problematic state is never entered.

The upshot is a clean separation. The *runtime* operations carry no path: Ins needs only its immediate anchor, Del only its target, and the merge recovers position from the LCA (§2). The path exists solely to discharge the framework's universally-quantified commutation VC over states that no execution produces. It is ghost state — read by the proof, erased at run time.

5 The asymmetry: Ins needs the path, Del does not

Strikingly, only the *insert* needs the path. A delete can always recompute its target from the live state, because at the moment a delete runs, the node it reparents *to* (its target's parent) is still present. Figure 7 contrasts the two: the delete reads its answer from a node that is alive *now*; the insert may need a node that was alive *then* but is gone *now*.

The intuition is temporal: a delete reparents *all current children* the instant it runs, keeping the forest connected; a concurrent (or reordered) insert is a child that was *not present* at that instant, so it missed the handoff and must supply its own ancestry.

6 Why no unconditioned proof exists

The path fixes the counterexample of §3; it is natural to hope the fixed design then discharges the standard framework schema as-is — the flat verification conditions, in which *commutation is an equation over all states*: for every concurrent pair left unordered by the reconciliation policy rc, $\mathsf{do}(\mathsf{do}(\sigma, a), b) = \mathsf{do}(\mathsf{do}(\sigma, b), a)$ for *every* σ, with the policy itself constrained (every non-commuting concurrent pair must be rc-oriented, and rc-edges must not chain). That hope fails twice, in instructive ways, and the failures are why this data type is verified through the *conditioned* framework instead.

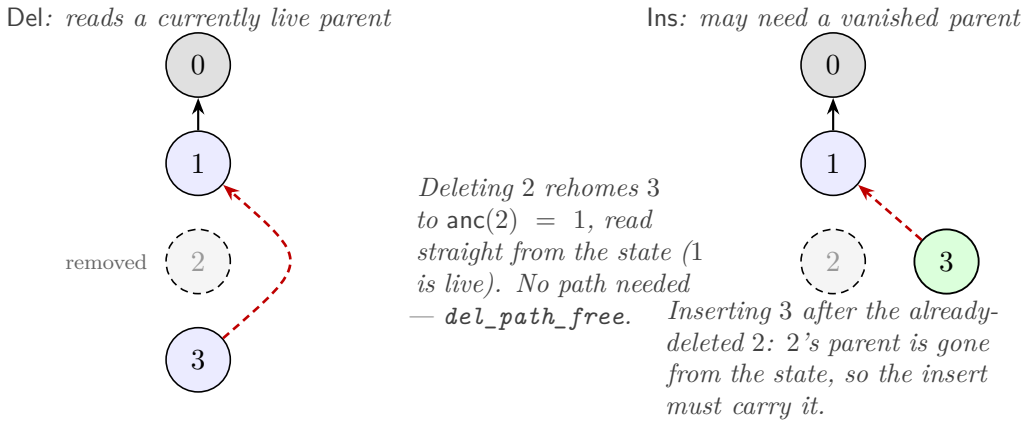


Figure 7: The asymmetry. Del hands off children to a parent that is still live at delete time, so it reads from state; a late Ins references a node that may have been spliced away, so it must carry the path. (`del_path_free` / `prefixfree_diverges`.)

Attempt 1: no path at all (the splice predecessor). The original tombstone-free design was the path-free splice RGA (`RGA_Splice`, parked on branch `wip/rga-splice`): an Ins carried only its immediate anchor, a Del only its target, and delete physically spliced the node out. Physical splice destroys the deleted node's parent pointer, so Ins-after-x and Del(x) do not commute; whether the insert ran before the splice decides whether it gets rehomed onto anc(x) or left dangling. The schema then *forces* an rc-edge ordering the pair — which makes the ordered-pair commutation obligation (`cond_comm_base`) non-vacuous, and that obligation is refuted by a three-operation trace (Figure 8).

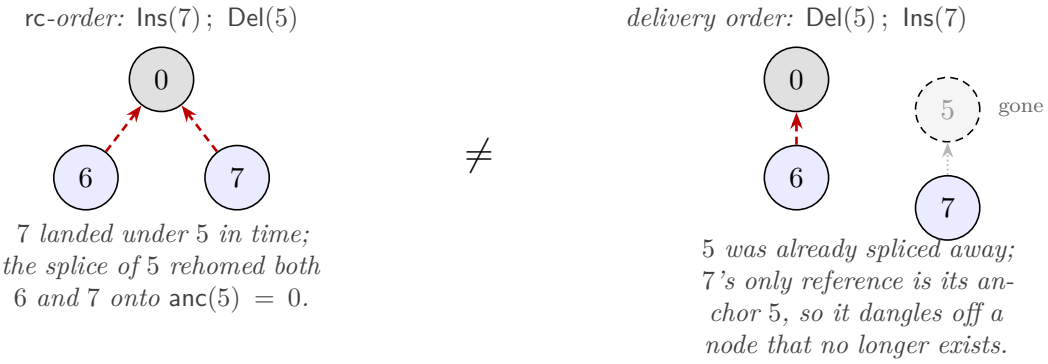


Figure 8: **The splice design fails the schema.** Start from root 0, node 5 under 0, node 6 under 5 (an earlier Ins-after-5). The concurrent pair Ins(7 after 5) ∥ Del(5) does not commute, so the schema forces an rc-edge (insert first). But replaying the mandated order against the delivery order diverges at 7: rehomed onto the root on the left, dangling on the right. `cond_comm_base` is false for this design (branch `wip/rga-splice`); no assignment of rc rescues it.

Attempt 2: the path — which commutes only on truthful states. The path-carrying design of this note repairs the algebra, but look at the fine print of the mechanised commutation fact (`rc_non_comm'`, §7): it holds *on accurate, fresh-id, root-not-stored states*. Those hypotheses are not decoration. The unconditioned schema quantifies over *all* states and all operation payloads

— including operations whose recorded path *lies* — and there the equation is false (Figure 9): while the anchor is alive the path is ignored (the insert reads the state), and once the anchor is deleted the path is *trusted* (the insert climbs the record); if record and history disagree, the two orders read two different sources of truth and diverge. No path discipline can be checked by do locally — whether a path is accurate is a property of the *execution* that produced it, not of the state it is applied to.

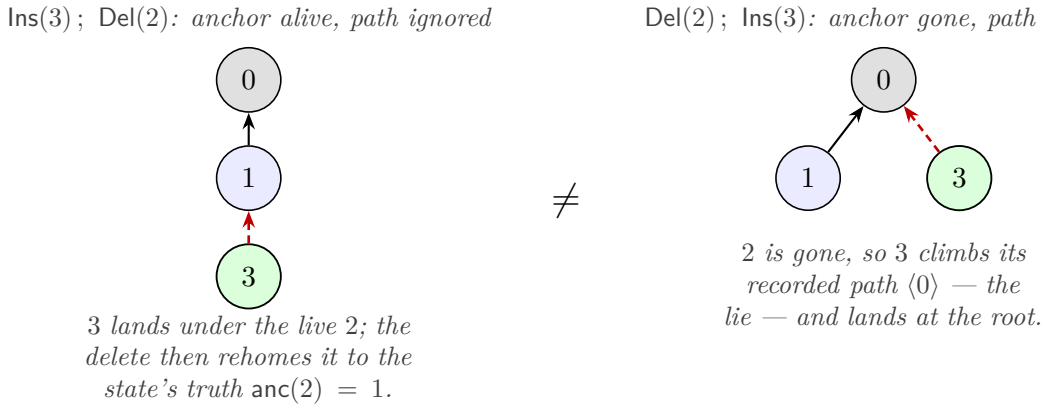


Figure 9: **A lying path breaks unconditioned commutation.** State: 1 under 0, 2 under 1. The operation Ins(3 after 2) carries the *inaccurate* path ⟨0⟩ (claiming 2 hangs off the root). Left order: the path is never consulted and 3 ends under 1. Right order: the path is consulted and 3 ends under 0. The schema's commutation equation quantifies over such states and payloads, so it is unprovable; the mechanised `rc_non_comm'` must and does assume accuracy.

Why this cannot be patched inside the schema. Two escape routes suggest themselves, and both are closed. *Weakening the VC* — baking "accurate, fresh, root-free" into the commutation statement — silently changes the schema: the framework's soundness theorem was proved for the original VCs, so the modified bundle discharges nothing until that soundness proof is redone (this project's costliest lesson; the redone proof *is* the conditioned framework). *Re-engineering the operations* — finding some tombstone-free design whose operations commute unconditionally — is refuted in general: `RGA_PrefixFree_Impossible.lean` shows no tombstone-free, prefix-free, local design can satisfy the rc = Either commutation VC, and ordering the insert/delete pair instead founders on rehoming non-transitivity (`RGA_DepComp_Gate`).

The resolution. The side conditions are promoted to first-class citizens: an *invariant* on states, an *applicability* discipline on operation generation (a well-behaved replica only issues an Ins whose recorded path is its actual view — which is all any honest replica can do), and commutation proved *conditioned* on both, propagated along reachability under honest delivery. That is the conditioned MRDT framework (`Sal/ConditionedMRDTs/`; companion note `mrdt-metatheory.pdf`), and this data type is its founding instance: the end-to-end theorem (`rga_tombstone_free_ra_linearizable3_eq`) is RA-linearizability up to observational equivalence at every reachable configuration under honest delivery. The price of leaving the unconditioned schema is real — the tombstoned RGA discharges the flat VCs in ∼170 lines, while this data type's conditioned chain runs to ∼10,000 — and, by the results above, unavoidable.

7 What is mechanised

Every figure corresponds to a Lean fact in `Sal/MRDTs/RGA_Tombstone_Free/`:

- `prefix_converges` / `prefixfree_diverges` — Figures 4–5: with the path the pair commutes; drop it and the same accurate pair diverges. The *insert* path is necessary.
- `del_path_free` / `deldel_pathfree_converges` — Figure 7: the *delete* path is dispensable; a Del reads its reparent target $\mathsf{anc}(\sigma, x)$ from the live state.
- `rc_non_comm'` — the full commutation VC: on accurate, fresh-id, root-not-stored states, *every* operation pair commutes.
- `RGA_PrefixFree_Impossible.lean` — a parameterised theorem: no tombstone-free, prefix-free, local design can satisfy the $\mathsf{rc} = \mathsf{Either}$ commutation VC.
- Figure 8 — the splice predecessor's `do`-level non-commutation and refuted `cond_comm_base`: branch `wip/rga-splice` (`Sal/MRDTs/RGA_Splice/RGA_Splice_MRDT.lean`). Figure 9 is definitional (the climb trusts the record); its schema-level content is that `rc_non_comm'`'s accuracy hypothesis cannot be dropped, which is what forces the conditioned framework (§6).
- `RGA_PrefixFree_Impl.lean` — the runnable counterpart of Figures 3 and 4: its *merge* converges the concurrent case (LCA), yet its single-replica `do` diverges (`merge_converges_concurrent` vs. `single_replica_do_diverges`).

One honest caveat. The merge's climb is bounded by a fuel equal to the node id, so its convergence relies on *id-monotone* anchors ($\mathsf{anc}(t) < t$). That holds under monotone timestamp allocation but is *not* guaranteed by the forest shape alone; a non-monotone state can make climb run out of fuel under merge (`merge_breaks_wf` in `RGA_Reachability_Invariant.lean`). So Figure 3's convergence is a theorem about reachable, monotone states — the precise extra condition the soundness argument must carry.